In[160]:=

```
ClearAll["Global`*"];
```

In[161]:=

```
tau = 0.02;
Vrest = -70.;
Vthresh = -55.;
Vreset = -75.;
R = 1.;

Iinput[t_?NumericQ] := If[0.05 ≤ t ≤ 0.4, 18., 0.];

Print["Parameters defined."]
```

```
Parameters defined.
```

In[168]:=

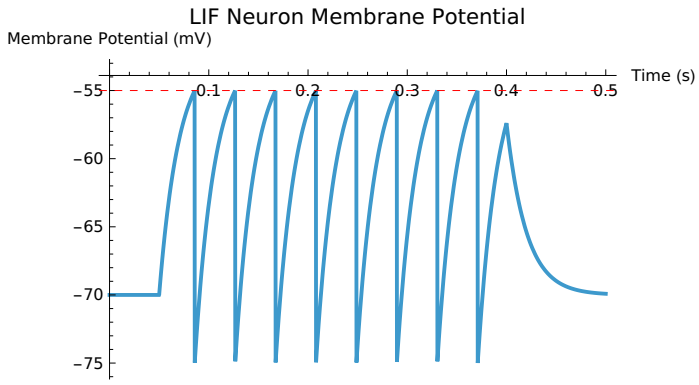
```
{sol, spikeTimes} = Reap[
 NDSolve[
  {
   V'[t] == (-(V[t] - Vrest) + R*Iinput[t])/tau,
   V[0] == Vrest,
   WhenEvent[V[t] ≥ Vthresh, {V[t] → Vreset, Sow[t]}]
  },
  V,
  {t, 0, 0.5}
 ]
];



spikeTimes = If[spikeTimes === {}, {}, spikeTimes[[1]]];
spikeCount = Length[spikeTimes];

Print["Neuron fired ", spikeCount, " spikes"];
Print["First spike times: ", Take[spikeTimes, UpTo[5]]];
```

```
Neuron fired 8 spikes
First spike times: {0.0858352, 0.126573, 0.16731, 0.208048, 0.248786}
```

In[173]:=

```
Plot[
 Evaluate[V[t] /. sol],
 {t, 0, 0.5},
 PlotRange → All,
 AxesLabel → {"Time (s)", "Membrane Potential (mV)"},
 PlotLabel → "LIF Neuron Membrane Potential",
 Epilog → {Red, Dashed, InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]}
]
```

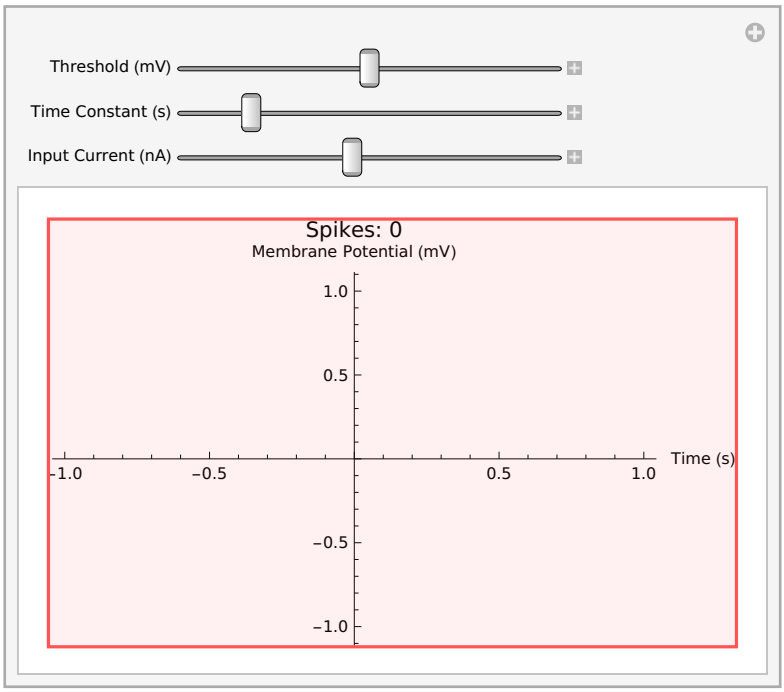
Out[173]=



In[174]:=

```
Manipulate[
 Module[{sol, spikeTimes, spikeCount, IinputLocal},

  IinputLocal[t_?NumericQ] := If[0.05 ≤ t ≤ 0.4, currentVal, 0.0];

  {sol, spikeTimes} = Reap[
   NDSolve[
    {
     V'[t] == (-(V[t] - Vrest) + R * IinputLocal[t]) / tauVal,
     V[0] == Vrest,
     WhenEvent[V[t] ≥ thresh, {V[t] → Vreset, Sow[t]}]
    },
    V,
    {t, 0, 0.5}
   ]
  ];

  spikeTimes = If[spikeTimes === {}, {}, spikeTimes[[1]]];
  spikeCount = Length[spikeTimes];

  Plot[
   Evaluate[V[t] /. sol],
   {t, 0, 0.5},
   PlotRange → All,
   AxesLabel → {"Time (s)", "Membrane Potential (mV)"},
   PlotLabel → StringForm["Spikes: ``", spikeCount],
   Epilog → {Red, Dashed, InfiniteLine[{{0, thresh}, {1, thresh}}]}
  ]
 ],

 {{thresh, -55.0, "Threshold (mV)"}, -65, -45, 1},
 {{tauVal, 0.02, "Time Constant (s)"}, 0.005, 0.10, 0.005},
 {{currentVal, 18.0, "Input Current (nA)"}, 0, 40, 1}
]
```

Out[174]=


Threshold (mV)
Time Constant (s)
Input Current (nA)
Spikes: 0
Membrane Potential (mV)
1.0
0.5
−0.5
−1.0
−1.0
−0.5
0.5
1.0
Time (s)

In[175]:=

```
fiData = Table[
 Module[{sol, spikeTimes, IinputLocal, spikeCount},

  IinputLocal[t_?NumericQ] := If[0.05 ≤ t ≤ 0.4, Icurr, 0.0];

  {sol, spikeTimes} = Reap[
   NDSolve[
    {
     V'[t] == (-(V[t] - Vrest) + R * IinputLocal[t])/tau,
     V[0] == Vrest,
     WhenEvent[V[t] ≥ Vthresh, {V[t] → Vreset, Sow[t]}]
    },
    V,
    {t, 0, 0.5}
   ]
  ];

  spikeTimes = If[spikeTimes === {}, {}, spikeTimes[[1]]];
  spikeCount = Length[spikeTimes];

  {Icurr, spikeCount}
 ],
 {Icurr, 0, 35, 1}
];
```

```
fiData
```

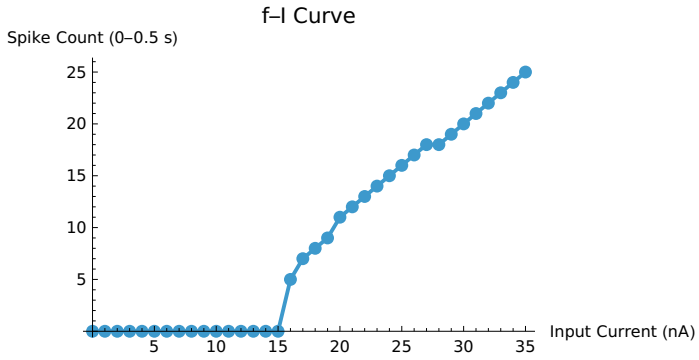
Out[176]=

```
{{0, 0}, {1, 0}, {2, 0}, {3, 0}, {4, 0}, {5, 0}, {6, 0}, {7, 0}, {8, 0}, {9, 0},
 {10, 0}, {11, 0}, {12, 0}, {13, 0}, {14, 0}, {15, 0}, {16, 5}, {17, 7}, {18, 8}, {19, 9},
 {20, 11}, {21, 12}, {22, 13}, {23, 14}, {24, 15}, {25, 16}, {26, 17}, {27, 18},
 {28, 18}, {29, 19}, {30, 20}, {31, 21}, {32, 22}, {33, 23}, {34, 24}, {35, 25}}
```

In[177]:=

```
ListLinePlot[
 fiData,
 AxesLabel → {"Input Current (nA)", "Spike Count (0–0.5 s)"},
 PlotLabel → "f–I Curve",
 PlotMarkers → Automatic
]
```

Out[177]=



1. At what input current does it first start firing?
Look at fiData and find the first current where spikeCount > 0.

Neuron first fires at ~__ nA.

2. Is it straight or curved? What would a straight line mean?

a. It's roughly linear in the middle range but not perfectly straight.

b. A perfectly straight line would mean spike output increases proportionally with input intensity (simple linear encoding).

3. Saturation at high currents?

It begins to flatten / keeps rising. In this basic LIF, there's no real refractory period unless you add one, so true biological saturation may not appear strongly.

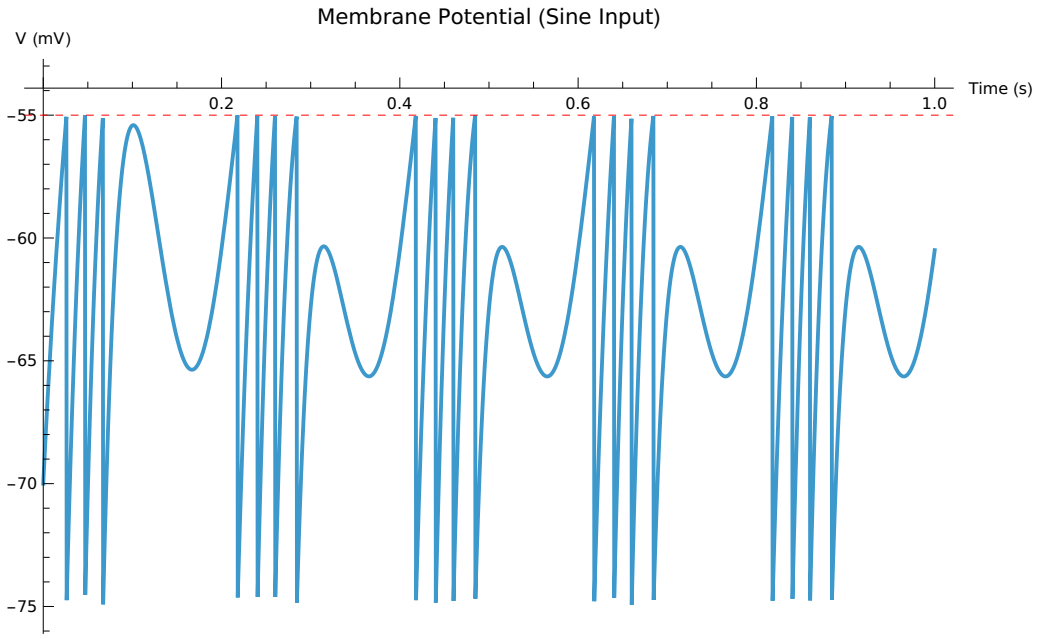
In[178]:=

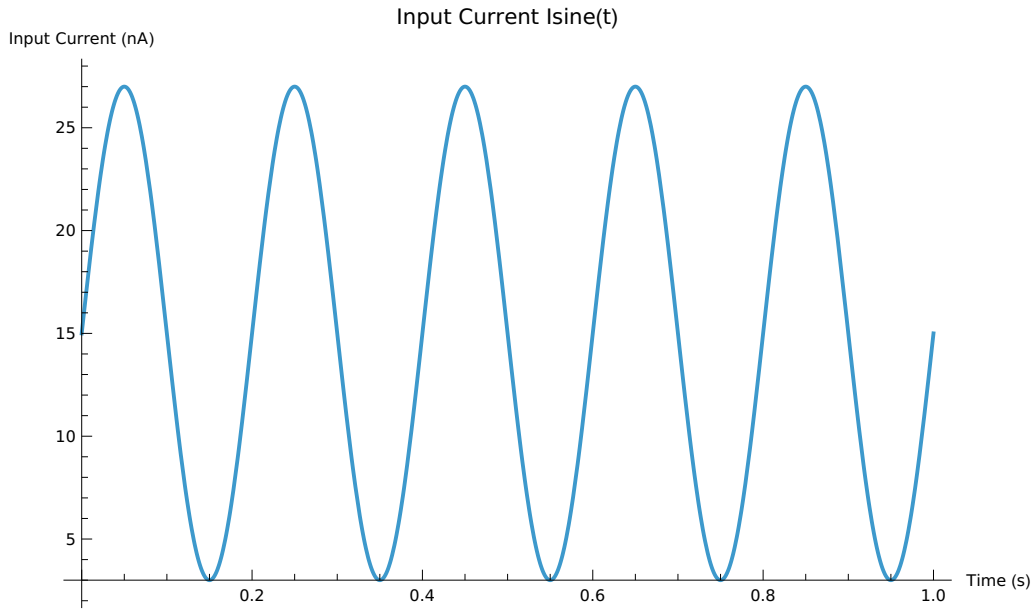
```
ClearAll[Isine, solSine, spikeTimesSine, spikeCountSine];


Isine[t_?NumericQ] := 15.0 + 12.0*Sin[2*Pi*5*t]; (* 5 Hz, ranges ~3 to 27 *)


{solSine, spikeTimesSine} = Reap[
 NDSolve[
  {
   V'[t] == (-(V[t] - Vrest) + R*Isine[t])/tau,
   V[0] == Vrest,
   WhenEvent[V[t] ≥ Vthresh, {V[t] → Vreset, Sow[t]}]
  },
  V,
  {t, 0, 1.0}
 ]
];


spikeTimesSine = If[spikeTimesSine === {}, {}, spikeTimesSine[[1]]];
spikeCountSine = Length[spikeTimesSine];

Print["Sine input spikes: ", spikeCountSine];
```

```
Sine input spikes: 19
```

In[184]:=

```
plotV = Plot[
 Evaluate[V[t] /. solSine],
 {t, 0, 1.0},
 PlotRange → All,
 AxesLabel → {"Time (s)", "V (mV)"},
 PlotLabel → "Membrane Potential (Sine Input)",
 Epilog → {Red, Dashed, InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]}
];



plotI = Plot[
 Isine[t],
 {t, 0, 1.0},
 PlotRange → All,
 AxesLabel → {"Time (s)", "Input Current (nA)"},
 PlotLabel → "Input Current Isine(t)"
];



GraphicsColumn[{plotV, plotI}]
```





Part D Reflection

1. The spike train roughly follows the sine wave: higher input → higher spike density.

2. Preserved: timing/density info about when input is strong.

3. Lost: exact analog amplitude (spikes are discrete; you don’t get a perfect continuous copy).

In[187]:=

```
threshData = Table[
 Module[{sol, spikeTimes, spikeCount},
 {sol, spikeTimes} = Reap[
  NDSolve[
   {
    V'[t] == (-(V[t] - Vrest) + R*Iinput[t])/tau,
    V[0] == Vrest,
    WhenEvent[V[t] ≥ thr, {V[t] → Vreset, Sow[t]}]
   },
   V,
   {t, 0, 0.5}
  ]
 ];
 spikeTimes = If[spikeTimes === {}, {}, spikeTimes[[1]]];
 spikeCount = Length[spikeTimes];
 {thr, spikeCount}
 ],
 {thr, -65, -45, 5}
];
```

```
Grid[Prepend[threshData, {"Threshold (mV)", "Spike Count (0-0.5s)"}], Frame → All]
```

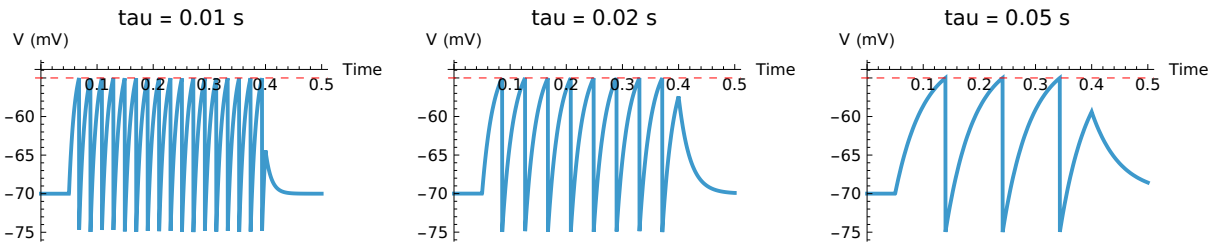
Out[188]=

Threshold (mV)	Spike Count (0-0.5s)
-65	31
-60	16
-55	8
-50	0
-45	0

In[189]:=

```
makePlotForTau[tauTest_] := Module[{solLocal},
 solLocal = NDSolve[
  {
   V'[t] == (-(V[t] - Vrest) + R*Iinput[t])/tauTest,
   V[0] == Vrest,
   WhenEvent[V[t] ≥ Vthresh, V[t] → Vreset]
  },
  V,
  {t, 0, 0.5}
 ];

 Plot[
  Evaluate[V[t] /. solLocal],
  {t, 0, 0.5},
  PlotRange → All,
  PlotLabel → StringForm["tau = `` s", tauTest],
  AxesLabel → {"Time", "V (mV)"},
  Epilog → {Red, Dashed, InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]}
 ]
];



GraphicsRow[
 {makePlotForTau[0.010], makePlotForTau[0.020], makePlotForTau[0.050]},
 ImageSize → Full
]
```

Out[190]=



In[191]:=

```
runNoisyTrial[] := Module[{Inoisy, solN, spikeTimesN, spikeCountN},

 Inoisy[t_?NumericQ] := 18.0 + RandomReal[{-8, 8}]; (* new noise each evaluation *)

 {solN, spikeTimesN} = Reap[
  NDSolve[
   {
    V'[t] == (-(V[t] - Vrest) + R*Inoisy[t])/tau,
    V[0] == Vrest,
    WhenEvent[V[t] ≥ Vthresh, {V[t] → Vreset, Sow[t]}]
   },
   V,
   {t, 0, 0.5}
  ]
 ];

 spikeTimesN = If[spikeTimesN === {}, {}, spikeTimesN[[1]]];
 spikeCountN = Length[spikeTimesN];

 {spikeCountN,
  Plot[Evaluate[V[t] /. solN], {t, 0, 0.5},
   PlotRange → All,
   PlotLabel → StringForm["Noisy trial spikes: ``", spikeCountN],
   AxesLabel → {"Time", "V (mV)"},
   Epilog → {Red, Dashed, InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]}
  ]}
];



trial1 = runNoisyTrial[];
trial2 = runNoisyTrial[];
trial3 = runNoisyTrial[];

{trial1[[1]], trial2[[1]], trial3[[1]]}
```

NDSolve: Maximum number of 776005 steps reached at the point t == 0.000589309.

NDSolve: Maximum number of 387223 steps reached at the point t == 0.000297159.

In[1]:=
```
GraphicsColumn[{trial1[[2]], trial2[[2]], trial3[[2]]}]
```

Part: Part specification trial1[[2]] is longer than depth of object.

Part: Part specification trial2[[2]] is longer than depth of object.

Part: Part specification trial3[[2]] is longer than depth of object.

General: Further output of Part::partd will be suppressed during this calculation.

Out[1]=

Final Reflection

1) Increasing input current (below threshold → above)

As I increased the input current from low values, the membrane potential stayed near the resting potential because the leak term pulled it back toward Vrest. Once the current was high enough, the voltage rose faster and eventually crossed the threshold, which triggered a spike and reset. Past that point, increasing current produced more frequent spikes (higher firing rate) because the membrane reached threshold more quickly after each reset. The “start firing” point occurred at the smallest current that produced at least one spike in the 0.5 s window (my rheobase was about ___ nA). This

threshold behavior shows how neurons act like decision devices: small inputs may not cause an output, but once input pushes the system past a critical point, spiking begins reliably.

2) Shape of the f–I curve (linear or not?)

My f–I curve was not perfectly linear: it stayed at zero for low currents, then increased once current exceeded rheobase, and the increase was roughly linear over a middle range. The initial flat region is important biologically because it reflects the fact that weak stimuli may be ignored (no spikes). A perfectly straight line would mean the neuron converts stimulus intensity to spike output with a constant gain across all inputs, which is an idealized encoder. The curvature/threshold region shows that real neural encoding is often nonlinear, especially near the firing threshold where small changes in input can cause big changes in spike count.

3) Effect of membrane time constant τ

Changing τ changed how quickly the membrane voltage responded to input current. With a smaller τ, the voltage changed faster, meaning the neuron reacted more quickly to changes and could reach threshold sooner (depending on parameters), often altering spike timing and sometimes increasing spike count within a fixed window. With a larger τ, the membrane integrated input more slowly and produced a smoother voltage trajectory, often delaying spikes or reducing spike count in the same time interval. Biologically, neurons with short time constants are better for fast, rapidly changing signals (high temporal precision), while neurons with long time constants behave more like integrators that average inputs over time (good for slower signals or sustained inputs).

4) LIF model vs real neurons: what it captures and what it misses

The LIF model captures key ideas: leak toward resting potential, integration of input, a threshold for spiking, and a reset after firing. It's useful because it reproduces basic spiking behavior and lets you test how parameters affect firing. However, it misses many real-neuron features: real neurons have multiple ion channels with nonlinear dynamics, refractory periods, spike-frequency adaptation, and complex dendritic structure that changes how inputs combine. It also does not model synapses and neurotransmitters explicitly, and it treats a spike as a simple threshold event rather than a full action potential waveform. So LIF is a simplified "decision-and-timing" model, not a biophysical model of the spike mechanism.

5) How Manipulate changed understanding + most surprising thing

Using the interactive sliders helped me understand the model more than just reading equations because I could immediately see how changing one parameter changed spike timing and spike count. The most surprising part was how sensitive firing rate is near threshold: small changes in threshold or

current could change the neuron from silent to rapidly spiking. Seeing the plot update in real time made it clear that spiking is not a gradual change—it often flips behavior once the system crosses a boundary. It also made the role of τ feel intuitive: larger τ looked like “smoothing” and slower response, while smaller τ looked like quicker changes in voltage.